Chapter 4: Classification

MATH 448: Statistical Learning — Problems 3, 6, 13, 15

Uzair Hamed Mohammed

October 16, 2026

Reproducibility for the applied questions: the script `solutions.py` computes every number and figure shown below.

```
pip install ISLP
python solutions.py          # computes results.json + figs/
typst compile solutions.typ  # produces this PDF
```

Contents

Problem 3 — Bayes classifier under QDA with p = 1 2
Problem 6 — Logistic regression on study hours and GPA 4
(a) Probability of an A 4
(b) Hours needed for a 50% chance of an A 4
Problem 13 — the Weekly data set 5
(a) Numerical and graphical summaries 5
(b) Logistic regression, all six predictors, full data 7
(c) Confusion matrix on the full data 8
(d)–(h) Five classifiers on Lag2, 2009–2010 held-out 8
(i) Best method on the held-out data 9
(j) Predictor combinations, transformations, interactions, and K 9
Problem 15 — Power functions and plots 11
(a) `Power()` 11
(b) `Power2(x, a)` 11
(c) Computing powers with `Power2()` 11
(d) `Power3(x, a)` returning a value 11
(e) Plot of $f(x) = x^2$ 11
(f) `PlotPower(x, a)` 12
Appendix — full `solutions.py` listing 14

Problem 3 — Bayes classifier under QDA with p = 1

Question. This problem relates to the QDA model, in which the observations within each class are drawn from a normal distribution with a class-specific mean vector and a **class-specific covariance matrix**. We consider the simple case where $p = 1$; i.e. there is only one feature. Suppose that we have K classes, and that if an observation belongs to the kth class then X comes from a one-dimensional normal distribution, $X \sim N(\mu_k, \sigma_k^2)$. Prove that in this case, the Bayes classifier is **not** linear. Argue that it is in fact **quadratic**.

Setup and discriminant function. Let π_k be the prior probability of class k. Under $Y = k$ the density of X is the one-dimensional normal density (equation (4.16) with a class-specific variance),

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x-\mu_k)^2\right),$$

and the Bayes classifier assigns x to the class k that maximizes the posterior

$$p_k(x) = \Pr(Y = k \mid X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^{K} \pi_l f_l(x)}.$$

Taking logarithms and dropping terms that do not depend on the class (the constant $-1/2 \log 2\pi$ appears in every class), we obtain the discriminant function

$$\begin{aligned}\delta_k(x) &= \log \pi_k - \log \sigma_k - \frac{(x-\mu_k)^2}{2\sigma_k^2} \\ &= -\frac{x^2}{2\sigma_k^2} + \left(\frac{\mu_k}{\sigma_k^2}\right)x - \frac{\mu_k^2}{2\sigma_k^2} - \log \sigma_k + \log \pi_k.\end{aligned}$$

Observe that $\delta_k(x)$ is a **quadratic function of x**: the coefficient of x^2, namely $-1/(2\sigma_k^2)$, depends on the class. Only when σ_k^2 does not depend on k does the x^2 term become common to all classes and cancel from $\delta_k - \delta_l$, so the decision **boundary** is linear; δ_k itself remains quadratic.

The boundary solves a quadratic equation. The Bayes decision rule switches from class k to class l exactly where the posteriors tie, i.e. where $\delta_k(x) = \delta_l(x)$. Writing out the equality,

$$\log \pi_k - \log \sigma_k - \frac{(x-\mu_k)^2}{2\sigma_k^2} = \log \pi_l - \log \sigma_l - \frac{(x-\mu_l)^2}{2\sigma_l^2},$$

then expanding the squares and collecting powers of x,

$$\begin{aligned}0 = \underbrace{\frac{1}{2}\left(\frac{1}{\sigma_k^2} - \frac{1}{\sigma_l^2}\right)}_{A} x^2 + \underbrace{\frac{\mu_l}{\sigma_l^2} - \frac{\mu_k}{\sigma_k^2}}_{B} x \\ + \underbrace{\frac{\mu_k^2}{2\sigma_k^2} - \frac{\mu_l^2}{2\sigma_l^2} - \log\left(\frac{\pi_k \sigma_l}{\pi_l \sigma_k}\right)}_{C}.\end{aligned}$$

This is a **quadratic equation in x**, $Ax^2 + Bx + C = 0$, whose leading coefficient

$$A = \frac{1}{2}\left(\frac{1}{\sigma_k^2} - \frac{1}{\sigma_l^2}\right)$$

vanishes if and only if $\sigma_k^2 = \sigma_l^2$. Under QDA's class-specific variances, generically $A \neq 0$, so the equation is genuinely quadratic:

- **Up to two boundary points.** The equation then has (generically) two distinct real roots $x_1 < x_2$, so the decision rule flips classes twice: the class with the **larger** variance dominates at $\pm\infty$ (its density spreads out more, so far in either tail $f_k > f_l$ for wide k), while the tighter class is selected only on the bounded interval between the two roots. The boundary therefore consists of **two points** rather than a single threshold — a configuration no linear classifier in one dimension could produce. A linear (LDA-style) boundary arises exactly in the degenerate sub-case $A = 0$, i.e. forcing $\sigma_k^2 = \sigma_l^2$.
- **Degenerate special case.** If the discriminant $B^2 - 4AC \leq 0$, the quadratic never changes sign, the tighter-variance class is **never** chosen on a bounded interval, and the classifier is constant — itself a behavior that unequal variances allow but equal variances (with unequal priors aside) do not.

Hence for the QDA model with $p = 1$, log-posterior ratios are quadratic functions of x and the Bayes decision boundaries are the roots of a quadratic: **the Bayes classifier is not linear; it is quadratic.**

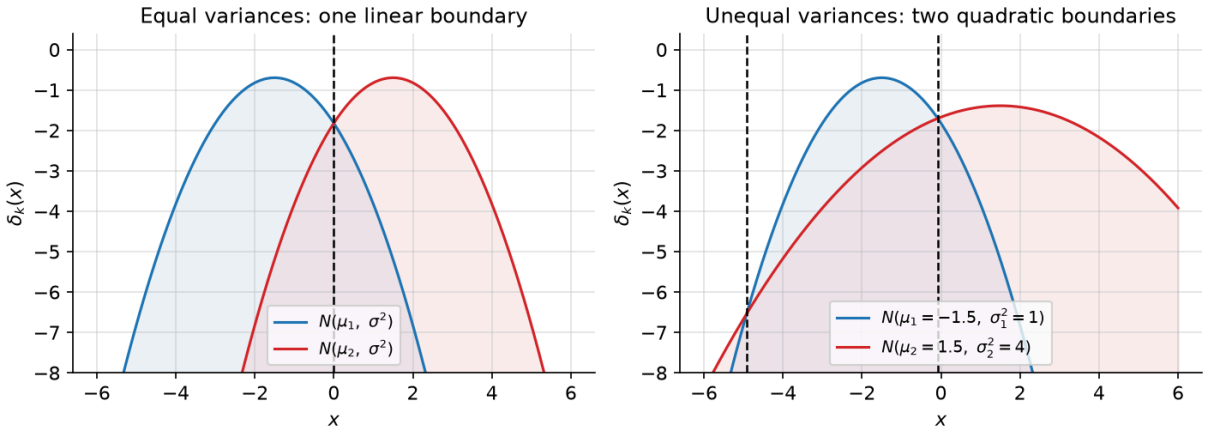


Figure 1: Illustration for $p = 1$, $K = 2$, equal priors $\pi_1 = \pi_2 = 0.5$, showing the log-posteriors $\delta_k(x)$. **Left:** equal variances (LDA-like): one linear boundary at $x = 0$. **Right:** class-specific variances $\sigma_1^2 = 1$, $\sigma_2^2 = 4$ (QDA): the boundaries are the two dashed lines $x \approx -4.92$ and $x \approx -0.08$, the roots of $0.375x^2 + 1.875x + 0.1506 = 0$ (equivalently, $\delta_1 - \delta_2 = 0$). `solutions.py` prints the residual and asserts that $\delta_1 - \delta_2$ vanishes at these two roots to machine precision.

Problem 6 — Logistic regression on study hours and GPA

> **Question.** We collect data on a statistics class with X_1 = hours studied, X_2 = undergrad GPA and Y = receives an A. The fitted logistic regression gives $\hat{\beta}_0 = -6$, $\hat{\beta}_1 = 0.05$, $\hat{\beta}_2 = 1$. (a) Estimate the probability that a student who studies for 40 h and has an undergrad GPA of 3.5 gets an A. (b) How many hours would that student need to study to have a 50% chance of getting an A?

(a) Probability of an A

The fitted logit for a student with $x_1 = 40$ hours and $x_2 = 3.5$ GPA is

$$\hat{\eta} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 = -6 + (0.05)(40) + (1)(3.5) = -6 + 2 + 3.5 = -0.5.$$

Applying the logistic function,

$$\hat{p} = \Pr(Y = \text{A} \mid X) = \frac{e^{-0.5}}{1 + e^{-0.5}} = \frac{1}{1 + e^{0.5}} = \frac{1}{1 + 1.6487} = 0.3775. \cdot$$

The estimated probability that this student receives an A is **0.3775**, i.e. about **37.75%**.

(b) Hours needed for a 50% chance of an A

$\hat{p} = 0.5$ means even odds, so the fitted logit must vanish:

$$\hat{\eta} = -6 + 0.05h + (1)(3.5) = 0 \quad \Longrightarrow \quad 0.05h = 2.5 \quad \Longrightarrow \quad h = 50. \cdot$$

So the student would need to study **50 hours**. Check: with $h = 50$, $\hat{\eta} = -6 + 2.5 + 3.5 = 0$, and indeed $\hat{p} = \frac{1}{1+e^0} = 0.5$ exactly — verified numerically in `solutions.py`.

Problem 13 — the Weekly data set

> **Questions.** Using the `Weekly` data set (1,089 weekly percentage returns of the S&P 500, 1990–2010): (a) produce numerical and graphical summaries and comment on any patterns; (b) fit a logistic regression of `Direction` on the five lag variables plus `Volume` using the full data, and identify any statistically significant predictors; (c) compute the confusion matrix and the overall fraction of correct predictions, and interpret the types of mistakes; (d) fit logistic regression on a 1990–2008 training set with `Lag2` as the only predictor, and compute the confusion matrix and fraction of correct predictions on the 2009–2010 held-out data; (e) repeat with LDA; (f) repeat with QDA; (g) repeat with KNN with $K = 1$; (h) repeat with naive Bayes; (i) which method appears best; (j) experiment with different predictor combinations (including transformations and interactions) and with values of K for KNN, and report the best combination found.

The data is loaded with `ISLP.load_data("Weekly")`: 1,089 observations, 9 variables (`Year`, `Lag1`–`Lag5`, `Volume`, `Today`, `Direction`), with 605 weeks going Up and 484 Down. The majority class ("always predict Up") would be correct 55.6% of the time. All models in (d)–(j) train on 1990–2008 and predict the 104 held-out weeks of 2009–2010.

(a) Numerical and graphical summaries

Summary statistics (lag variables and `Today` in percent, `Volume` in billions of weekly shares):

Statistic	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today
Mean	2000.0	0.151	0.151	0.147	0.146	0.140	1.5746	0.150
Std. dev.	6.0	2.357	2.357	2.361	2.360	2.361	1.6866	2.357
Minimum	1990.0	-18.195	-18.195	-18.195	-18.195	-18.195	0.0875	-18.195
Maximum	2010.0	12.026	12.026	12.026	12.026	12.026	9.3282	12.026

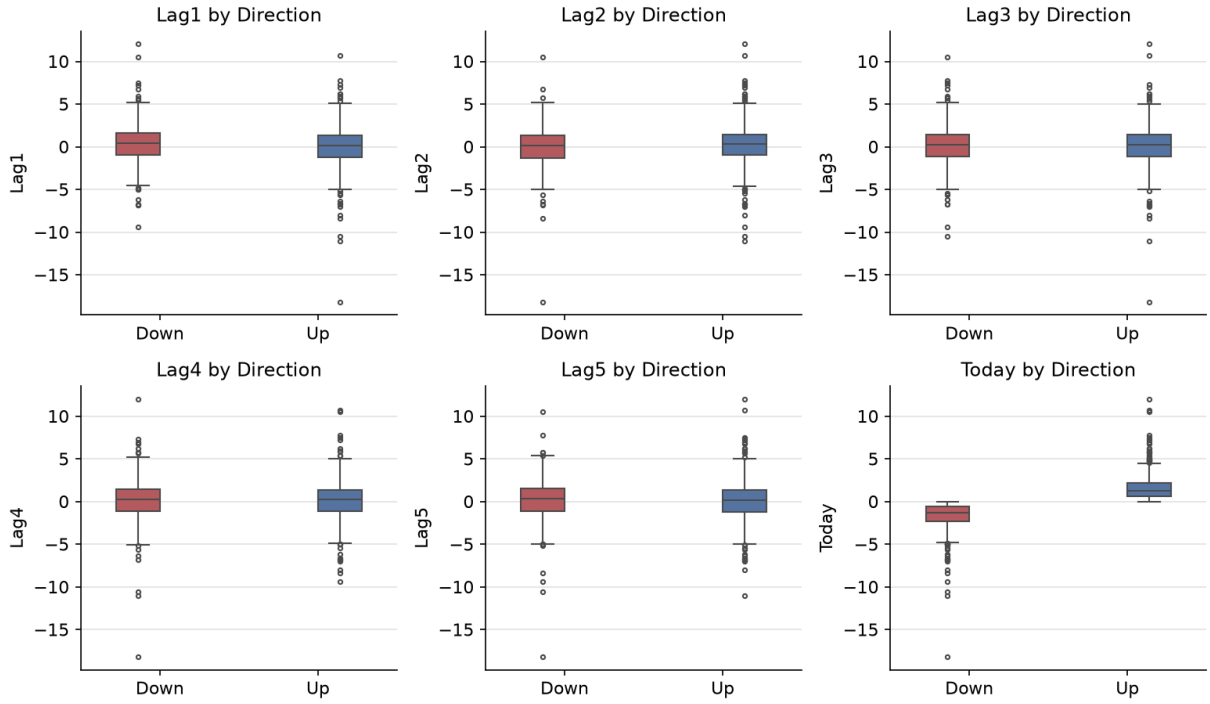


Figure 2: Boxplots of `Lag1`–`Lag5` and `Today` split by `Direction` (Down = red, Up = blue). The lag distributions for the two classes overlap almost completely, with only small median shifts; `Today` separates the classes almost perfectly, by construction.

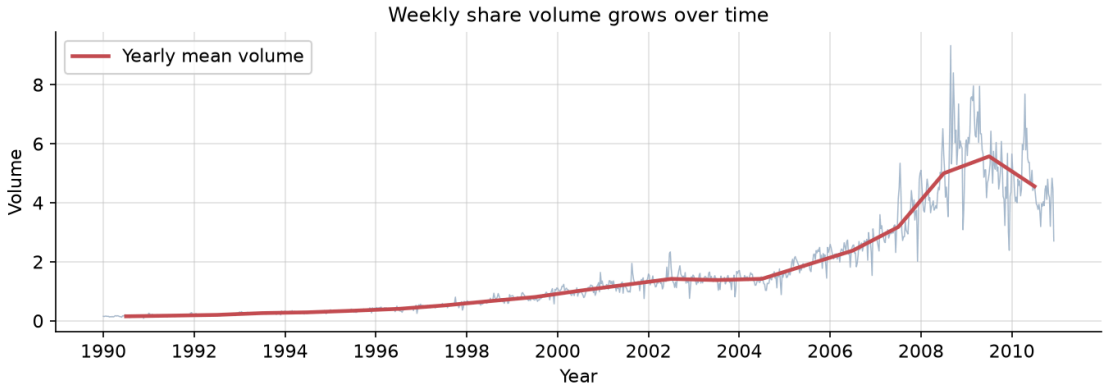


Figure 3: Weekly `Volume`, 1990–2010 (light) with the yearly mean (red). Volume trends strongly upward until the 2008–2009 financial crisis and declines afterwards.

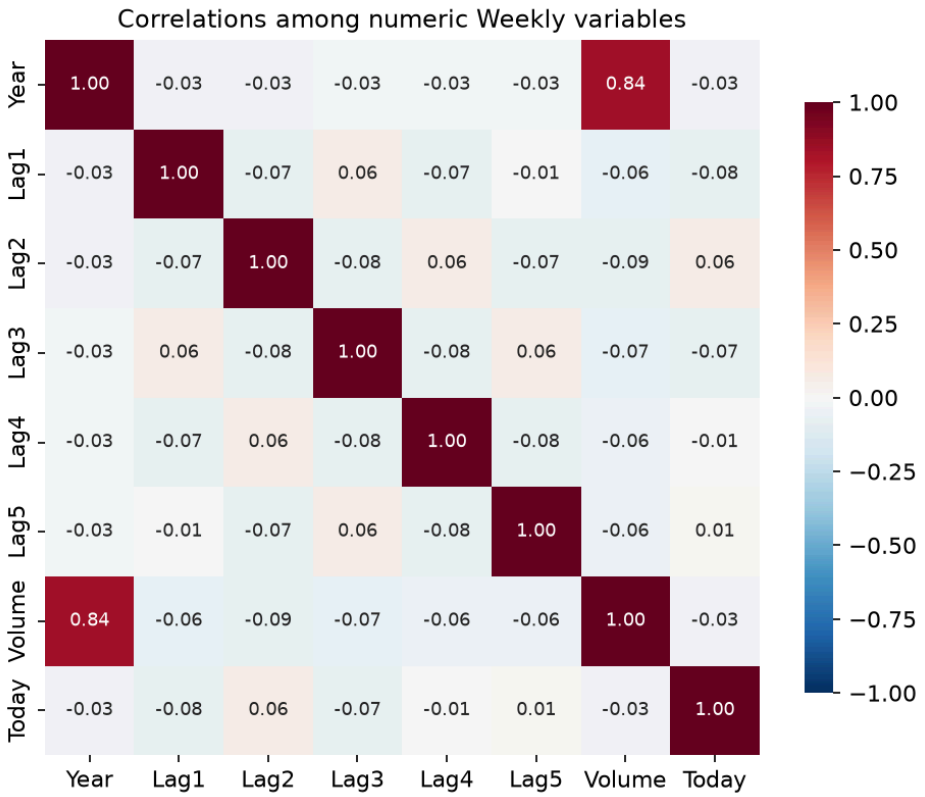


Figure 4: Correlations among the numeric variables. The only substantial correlation is `Year`–`Volume` ($r = 0.84$); the lag variables are mutually almost uncorrelated ($|r| \leq 0.09$) and only weakly correlated with `Today` ($|r| \leq 0.12$).

Patterns.

1. **Volume grows over time:** mean weekly volume rises from roughly 0.15 billion shares (1990) to about 1.0 by 2000 and 5.0–5.6 by 2008–2009, then falls to 4.6 in 2010. This agrees with the strong `Year`–`Volume` correlation of $r = 0.84$, and the growth looks roughly exponential up to 2008–2009.
2. **The lag variables are essentially mutually uncorrelated** (all $|r| \leq 0.09$), so multicollinearity among lags is negligible, and the plots show no visible structure among the five lags.
3. **Weak univariate signal:** boxplots of each lag by `Direction` overlap almost fully; e.g. the median of `Lag1` is shifted by only a fraction of a percentage point between Down and Up weeks. Any classifier built only on the lags is therefore expected to hover close to the 55.6% "always Up" baseline.
4. `Direction` is essentially the sign of `Today`, so `Today` separates the classes almost perfectly in the boxplot — but `Today` is not available at prediction time and is excluded from every fitted model.

(b) Logistic regression, all six predictors, full data

Fitting a logistic regression (binomial GLM) of `Direction` (Up = 1) on `Lag1`–`Lag5` and `Volume` using all 1,089 weeks:

Term	Coef.	Std. err.	z	p-value
Intercept	0.2669	0.0859	3.1056	**0.0019**
Lag1	−0.0413	0.0264	−1.5626	0.1181
Lag2	0.0584	0.0269	2.1754	**0.0296**
Lag3	−0.0161	0.0267	−0.6024	0.5469

Term	Coef.	Std. err.	z	p-value
Lag4	−0.0278	0.0265	−1.0501	0.2937
Lag5	−0.0145	0.0264	−0.5485	0.5833
Volume	−0.0227	0.0369	−0.6163	0.5377

Significant predictors. At the 5% level, only `Lag2` is statistically significant ($\hat{\beta}_2 = 0.0584$, $p = 0.0296$, positive coefficient: a higher return two weeks ago is associated with a higher probability of an Up week). The intercept is also significant ($p = 0.0019$), mainly reflecting the Up/Down class imbalance rather than any predictor. `Lag1` is borderline ($p = 0.1181$), while `Lag3`, `Lag4`, `Lag5`, and `Volume` are far from significant ($p \geq 0.29$).

(c) Confusion matrix on the full data

Predicting on the training data (fitted probability $> 0.5 \rightarrow$ "Up"):

Actual \ Predicted	Down	Up
Down	54	430
Up	48	557

Training fraction correct: 611 / 1089 = **56.11%** — barely above the 55.6% "always Up" baseline.

Types of mistakes: the classifier is extremely lopsided. Of its 1089 predictions, only 54 + 48 = 102 are Down; it correctly flags just 54 of the 484 true Down weeks (11.2%) while its Up predictions are right on 557 of 605 weeks (92.1%). The dominant error is a **false positive for Up**: the market is predicted to go Up when it actually goes Down (430 such weeks), versus only 48 of the opposite type. Most fitted probabilities sit just above 0.5, so the 0.5 rule predicts Up for 987 of the 1089 weeks, and logistic regression here behaves like an "always Up" classifier and misses almost every down week.

(d)–(h) Five classifiers on Lag2, 2009–2010 held-out

Each method is trained on 1990–2008 with `Lag2` as the sole predictor; test accuracy is computed on the 2009–2010 weeks. The table columns are (actual Down → predicted Down), (actual Down → predicted Up), (actual Up → predicted Down), and (actual Up → predicted Up):

Method	D→D	D→U	U→D	U→U	% correct
Logit	9	34	5	56	62.5%
LDA	9	34	5	56	62.5%
QDA	0	43	0	61	58.7%
KNN	22	21	32	29	49%
NB	0	43	0	61	58.7%

Part (d) — **logistic regression:** 65/104 = **62.5%** correct; it catches 9 of 43 true Down weeks (matrix row 1) and 56 of 61 true Up weeks. Part (e) — **LDA:** identical to logistic regression here, 65/104 = **62.5%** — with a single predictor both reduce to a single threshold, and on this split they select the same one; the confusion matrix is exactly the same. Part (f) — **QDA:** 61/104 = **58.65%**; the fitted quadratic `Lag2` boundary has no real roots (discriminant $\approx -3.1 \times 10^{-3}$; complex roots $\approx -2.71 \pm 2.70i$), so it predicts Up for **all 104** test weeks (matrix [[0, 43], [0, 61]]) — i.e. it degenerates to the "always Up" rule. Part (g) — **KNN, $K = 1$:** 51/104 = **49.04%** — well below the 58.7% always-Up baseline on

these 104 weeks (61/104), and essentially chance; with a noisy single predictor and $K = 1$ the decision follows the nearest neighbor's realized direction essentially at random. Part (h) — **naive Bayes:** 61/104 = **58.65%** — with one predictor it fits essentially the same one-dimensional quadratic boundary as QDA and likewise predicts Up for every held-out week.

(i) Best method on the held-out data

Logistic regression and LDA are the best, tying at **62.5%** (65/104): both predict Up 90 of 104 weeks but catch 9 of the 43 true Down weeks. They are identical here because with a single one-dimensional predictor and fairly symmetric classes the linear boundary is the same. QDA and naive Bayes (58.65%) both predict Up for **every** held-out week, and KNN with $K = 1$ is no better than chance (49.04%). All methods are still far from reliable in this noisy financial setting.

(j) Predictor combinations, transformations, interactions, and K

I re-fit every method on combinations of predictors and transformations on the same 1990–2008 train / 2009–2010 test split: 15 predictor sets (including `Lag2^2`, `Lag1^2`, the interaction `Lag1*Lag2`, and `log(Volume)`) cross-fitted with the four regression-type methods plus KNN over $K \in \{1, 2, 3, 5, 7, 9, 10, 11, 12, 13, 14, 15, 17, 20, 25, 30, 47, 50\}$ — 330 fits in total. Test accuracies are tabulated below; for KNN, the reported entry is its best accuracy over the values of K tried (the winning K is shown in the last column).

Predictors	**Logit**	**LDA**	**QDA**	**NB**	**KNN**	***K***
Lag1 + Lag2 + Lag1² + Lag2²	0.567	0.577	0.644	0.577	0.596	50
Lag2	0.625	0.625	0.587	0.587	0.615	47
Lag1 + Lag2	0.577	0.577	0.558	0.538	0.625	25
Lag2 + Lag3	0.625	0.625	0.606	0.587	0.587	17
Lag2 + Lag2²	0.625	0.615	0.625	0.577	0.606	47
Lag1 + Lag2 + Lag3	0.577	0.577	0.548	0.538	0.615	13
Lag2 + Lag5	0.606	0.596	0.51	0.481	0.606	17
Lag1 + Lag2 + Volume	0.529	0.529	0.462	0.423	0.606	47
Lag2 + Volume	0.538	0.538	0.471	0.462	0.596	9
Lag2 + Lag2² + Lag1 · Lag2	0.577	0.577	0.481	0.587	0.567	5
Lag2 + log(Vol)	0.577	0.577	0.558	0.548	0.587	3
Lag1 + Lag2 + Lag1 · Lag2	0.577	0.577	0.462	0.471	0.577	25
Lag1–5 + Volume + Lag1 · Lag2	0.462	0.462	0.442	0.423	0.577	17
Lag1–5	0.548	0.548	0.462	0.481	0.567	10
Lag1–5 + Volume	0.462	0.462	0.433	0.452	0.558	9

Entries are test-set accuracy (fraction correct on 104 weeks). The overall best is highlighted.

Actual \ Predicted	**Down**	**Up**
Down	20	23
Up	14	47

Best combination found: QDA on `Lag1`, `Lag2`, `Lag1^2`, `Lag2^2` — 67 / 104 = **64.42%** correct, the only configuration above 63%. The squared terms let the quadratic class densities model asymmetric tail behavior around zero returns.

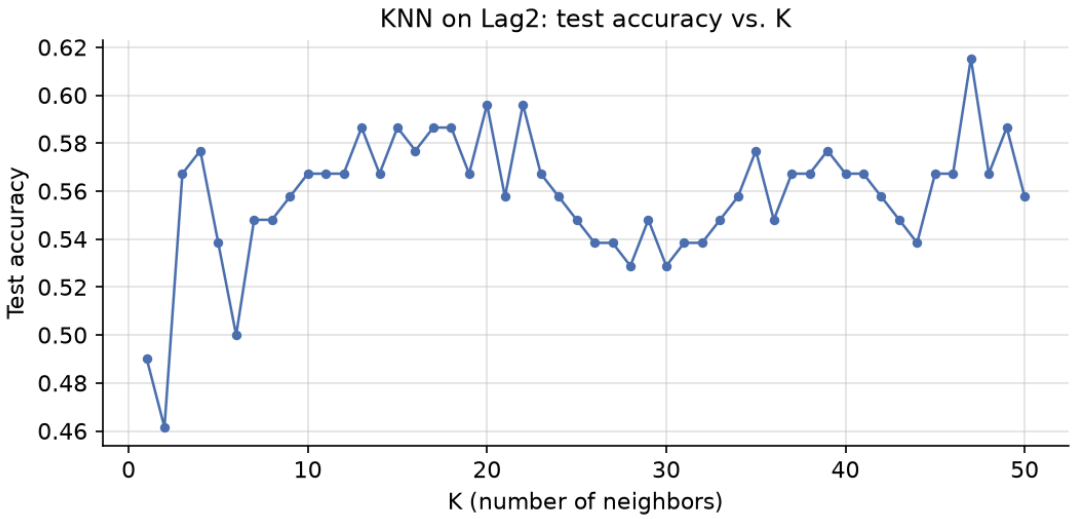


Figure 5: `Lag2`-only KNN test accuracy as a function of K. Accuracy is erratic ($K = 1$: 49.0%), stabilizes within 55–62% for most larger K, and the best single fit is $K = 47$ (61.5%); $K = 20$ gives 59.6%.

Interpretation. Including quadratic terms in `Lag1` and `Lag2` and using QDA pushes the held-out accuracy to 64.4%; several simpler alternatives sit at 62.5% — logistic or LDA on `Lag2`, logistic or LDA on `Lag2`+`Lag3`, logistic or QDA on `Lag2`+`Lag2^2`, and `Lag1`+`Lag2` with KNN at $K = 25$ (LDA on `Lag2`+`Lag2^2` is a notch lower at 61.5%). However, this comparison has strong selection bias: 330 models were fitted and evaluated on the same 104 held-out weeks, so the winner's edge (64.4% vs. 62.5%, i.e. two extra correct weeks) may largely reflect chance, and the single predictor `Lag2` with plain logistic regression or LDA remains the most defensible choice.

Problem 15 — Power functions and plots

> **Question.** (a) Write a function `Power()` that prints 2^3. (b) Create `Power2(x, a)` that prints x^a for any two numbers x and a. (c) Using `Power2()`, compute 10^3, 8^{17}, and 131^3. (d) Create `Power3(x, a)` that **returns** x^a. (e) Using `Power3()`, plot $f(x) = x^2$ for $x = 1, ..., 10$, labeling the axes and the title, optionally with a log-scale axis. (f) Create `PlotPower(x, a)` that plots x^a against x for a sequence of values of x; e.g. `PlotPower(np.arange(1, 11), 3)` plots $1, ..., 10$ against their cubes.

(a) `Power()`

```
def Power():
    print(2**3)

Power()   # output: 8
```

Calling `Power()` prints `8` (i.e., $2^3 = 8$).

(b) `Power2(x, a)`

```
def Power2(x, a):
    print(x**a)

Power2(3, 8)   # output: 6561
```

Calling `Power2(3, 8)` prints `6561`, the value of 3^8.

(c) Computing powers with `Power2()`

```
Power2(10, 3)    # output: 1000
Power2(8, 17)    # output: 2251799813685248
Power2(131, 3)   # output: 2248091
```

So $10^3 = \mathbf{1000}$, $8^{17} = \mathbf{2251799813685248}$, and $131^3 = \mathbf{2248091}$ (we also verify these with the returned-value version `Power3` so they can be stored programmatically).

(d) `Power3(x, a)` returning a value

```
def Power3(x, a):
    result = x**a
    return result
```

Unlike `Power2`, this returns the value as a Python object so it can be stored and reused, e.g. `x = Power3(3, 8)` yields `x == 6561`.

(e) Plot of $f(x) = x^2$

```
x10 = np.arange(1, 11)
fig, ax = plt.subplots(figsize=(6.4, 4.2))
ax.plot(x10, Power3(x10, 2), marker="o", lw=1.8)
ax.set_xlabel("x")
ax.set_ylabel("$x^2$")
ax.set_title("$f(x) = x^2$")
fig.tight_layout()
```

I additionally display the same curve on a log-log scale, discussed below.

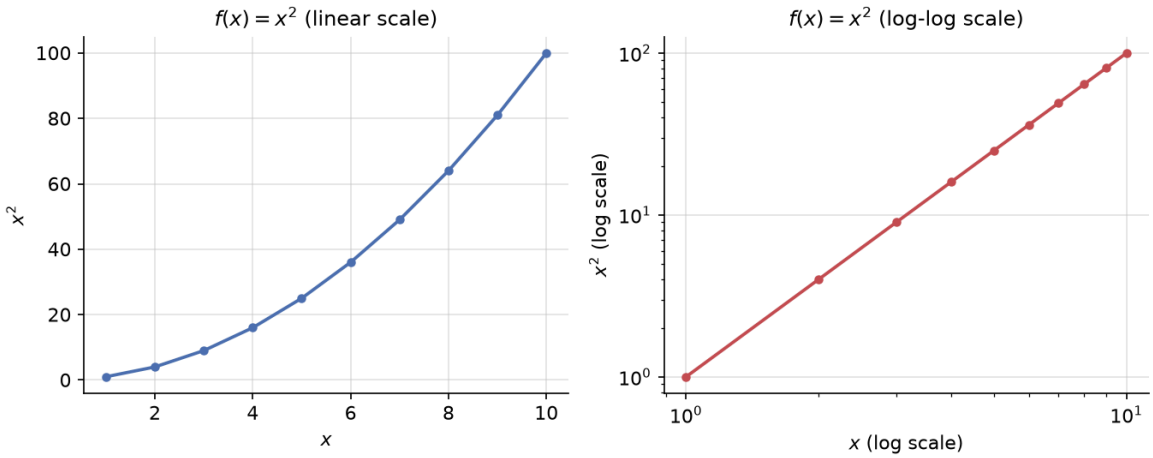


Figure 6: (e) Left: $f(x) = x^2$ on a linear scale for $x = 1, ..., 10$. Right: the same data with `ax.set_xscale("log")` and `ax.set_yscale("log")`; the parabola becomes a straight line with slope 2 on the log-log axes.

(f) `PlotPower(x, a)`

```
def PlotPower(x, a):
    fig, ax = plt.subplots(figsize=(6.4, 4.2))
    ax.plot(x, Power3(x, a), marker="o", color="#4c72b0", lw=1.8)
    ax.set_xlabel("x")
    if isinstance(a, int):
        ax.set_ylabel(rf"$x^{a}$")
    else:
        ax.set_ylabel(r"$x^a$")
    ax.set_title(rf"$y = x^{a}$")
    fig.tight_layout()
    return fig

PlotPower(np.arange(1, 11), 3)
```

Calling `PlotPower(np.arange(1, 11), 3)` produces Figure 7, with the x-axis showing $1, ..., 10$ and the y-axis showing $1^3, ..., 10^3$.

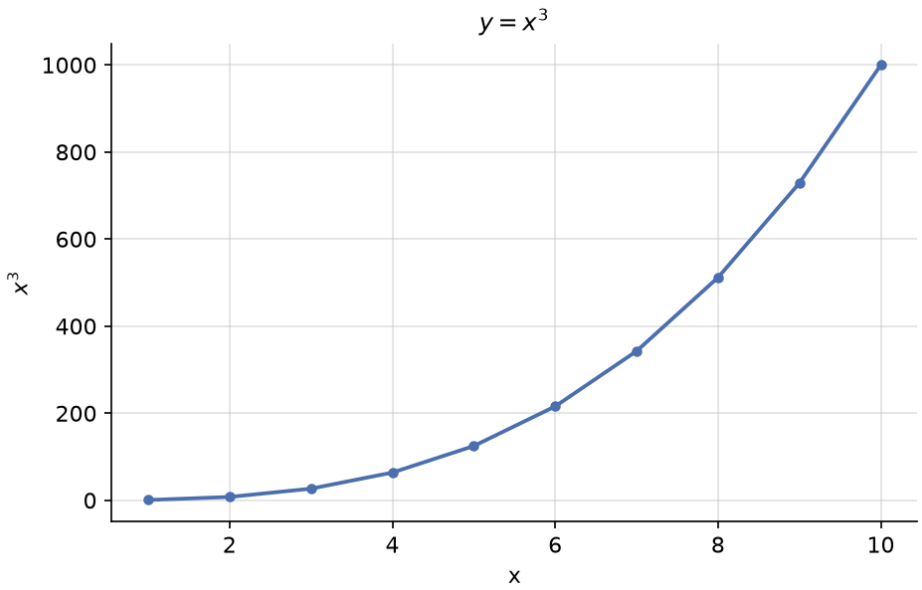


Figure 7: (f) `PlotPower(np.arange(1, 11), 3)`: x against x^3.

Appendix — full `solutions.py` listing

```python
# ruff: noqa
"""
MATH 448: Statistical Learning — Homework 3 (Chapter 4: Classification)
Problems 3, 6, 13, 15

Author:  Uzair Hamed Mohammed
Date:    10/16/2026

This script produces every numerical result and figure used in the solutions PDF.
Run:  python solutions.py
Outputs:  results.json  (all numeric tables / values)
          figs/*.png    (all figures)
"""

from __future__ import annotations

import json
import os

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
from ISLP import load_data
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis,
)
from sklearn.metrics import confusion_matrix
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

HERE = os.path.dirname(os.path.abspath(__file__))
FIGS = os.path.join(HERE, "figs")
os.makedirs(FIGS, exist_ok=True)

RNG = np.random.default_rng(0)  # single seeded generator for any randomness

plt.rcParams.update(
    {
        "figure.dpi": 125,
        "savefig.dpi": 150,
        "axes.grid": True,
        "grid.alpha": 0.35,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "font.size": 10.5,
        "axes.titlesize": 11.5,
    }
)

# All numeric results accumulate into this dict, dumped to results.json at the end.
RESULTS: dict = {}


def savefig(fig: plt.Figure, name: str) -> None:
    path = os.path.join(FIGS, name)
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)
    print(f"  [fig] {name}")


# ======================================================================
# Question 3 — QDA with p = 1 is quadratic, not linear
# ======================================================================
# Written proof goes in the PDF.  Here we illustrate it numerically:
# the Bayes boundary solves delta_1(x) = delta_2(x), which is quadratic
# whenever sigma_1^2 != sigma_2^2.  Equal variances give the single linear
# boundary (left panel); unequal variances give two roots (right panel).

MU1, S1SQ = -1.5, 1.0  # class 1: mu_1, sigma_1^2
MU2, S2SQ = 1.5, 4.0  # class 2: mu_2, sigma_2^2

xgrid = np.linspace(-6, 6, 1200)


def delta(x: np.ndarray, mu: float, var: float, logprior: float = 0.0) -> np.ndarray:
    """Log-posterior delta_k(x) for a N(mu, var) class density (p = 1)."""
    return logprior - 0.5 * np.log(var) - (x - mu) ** 2 / (2 * var)


def del_k_minus_del_l_squared_terms(
    mu1: float, v1: float, mu2: float, v2: float
) -> tuple[float, float, float]:
    """Coefficients (a, b, c) of the x^2, x, const terms of delta_1 - delta_2.
```

```
    delta_1 - delta_2 = -log(sigma_1/sigma_2) - 0.5 [ (x-mu_1)^2/v_1 - (x-mu_2)^2/v_2 ]
    After multiplying through by -1 (the equation delta_1 = delta_2 == a x^2 + b x + c = 0 with log terms folded into c):
      a = 0.5 (1/v1 - 1/v2)
      b = -(mu1/v1 - mu2/v2)
      c = 0.5 (mu1^2/v1 - mu2^2/v2)
    plus the prior/normalization constants, returned separately.
    """
    a = 0.5 * (1 / v1 - 1 / v2)
    b = -(mu1 / v1 - mu2 / v2)
    c = 0.5 * (mu1**2 / v1 - mu2**2 / v2)
    return a, b, c


def boundary_roots(
    mu1: float, v1: float, mu2: float, v2: float, log_prior_ratio: float = 0.0
) -> np.ndarray:
    """Roots of delta_1(x) = delta_2(x), equiprobable classes by default.

    Solves  a x^2 + b x + c - log(pi1/pi2) - (1/2) log(v2/v1) = 0,
    i.e. includes the variance-normalization term of the log posterior.
    """
    a, b, c = del_k_minus_del_l_squared_terms(mu1, v1, mu2, v2)
    c = c - log_prior_ratio - 0.5 * np.log(v2 / v1)
    return np.roots([a, b, c]) if abs(a) > 1e-12 else np.array([-c / b])


eq_roots = boundary_roots(MU1, 1.0, MU2, 1.0)
uneq_roots = boundary_roots(
    MU1, S1SQ, MU2, S2SQ, log_prior_ratio=float(np.log(0.5) - np.log(0.5))
)
_a, _b, _c = del_k_minus_del_l_squared_terms(MU1, S1SQ, MU2, S2SQ)
_c_full = _c - 0.5 * np.log(S2SQ / S1SQ)  # equal priors, variance-norm term kept
RESULTS["q3"] = {
    "equal_var_roots": [float(r) for r in eq_roots],
    "unequal_var_roots": [float(r) for r in uneq_roots],
    "a_quad": float(_a),
    "b_quad": float(_b),
    "c_quad_full": float(_c_full),
}

# Verify that delta_1 - delta_2 actually vanishes at the roots (machine precision).
_resid = [
    abs(float(delta(r, MU1, S1SQ, np.log(0.5)) - delta(r, MU2, S2SQ, np.log(0.5))))
    for r in uneq_roots
]
RESULTS["q3"]["boundary_residuals"] = _resid
print(f"Q3: max |delta_1 - delta_2| at boundary roots = {max(_resid):.2e}")
assert max(_resid) < 1e-12, "boundary roots do not solve delta_1 = delta_2"

fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.6))

# --- Panel A: equal variances -> linear boundary ---------------------
for mu, var, c, lab in [
    (MU1, 1.0, "#1f77b4", r"$N(\mu_1,\ \sigma^2)$"),
    (MU2, 1.0, "#d62728", r"$N(\mu_2,\ \sigma^2)$"),
]:
    d = delta(xgrid, mu, var, np.log(0.5))
    axes[0].plot(xgrid, d, color=c, label=lab)
    axes[0].fill_between(xgrid, -8, d, color=c, alpha=0.08)
for r in eq_roots:
    axes[0].axvline(r, color="k", ls="--", lw=1.2)
axes[0].set_title("Equal variances: one linear boundary")
axes[0].set_xlabel("$x$")
axes[0].set_ylabel(r"$\delta_k(x)$")
axes[0].set_ylim(-8, 0.4)
axes[0].legend(loc="lower center", fontsize=9)

# --- Panel B: unequal variances -> quadratic boundary ----------------
for mu, var, c, lab in [
    (MU1, S1SQ, "#1f77b4", rf"$N(\mu_1={MU1:g},\ \sigma_1^2={S1SQ:g})$"),
    (MU2, S2SQ, "#d62728", rf"$N(\mu_2={MU2:g},\ \sigma_2^2={S2SQ:g})$"),
]:
    d = delta(xgrid, mu, var, np.log(0.5))
    axes[1].plot(xgrid, d, color=c, label=lab)
    axes[1].fill_between(xgrid, -8, d, color=c, alpha=0.08)
for r in uneq_roots:
    axes[1].axvline(r, color="k", ls="--", lw=1.2)
axes[1].set_title("Unequal variances: two quadratic boundaries")
axes[1].set_xlabel("$x$")
axes[1].set_ylabel(r"$\delta_k(x)$")
axes[1].set_ylim(-8, 0.4)
axes[1].legend(loc="lower center", fontsize=9)

fig.tight_layout()
savefig(fig, "q3_qda_boundary.png")

# ======================================================================
# Question 6 — logistic regression probability estimates
# ======================================================================
# beta0 = -6, beta1 = 0.05 (hours), beta2 = 1 (GPA)   [Y = receive an A]

B0, B1, B2 = -6.0, 0.05, 1.0
```

```
def logit_to_prob(eta: float) -> float:
    return float(1 / (1 + np.exp(-eta)))


q6a = logit_to_prob(B0 + B1 * 40 + B2 * 3.5)
# 50% chance <=> logit = 0 <=> -6 + 0.05 h + 3.5 = 0  <=>  h = 50
q6b = (6.0 - B2 * 3.5) / B1
q6b_check = logit_to_prob(B0 + B1 * q6b + B2 * 3.5)
RESULTS["q6"] = {
    "eta_a": float(B0 + B1 * 40 + B2 * 3.5),
    "p_a": q6a,
    "hours_b": float(q6b),
    "p_b_check": q6b_check,
}
print(f"Q6: (a) p = {q6a:.4f};  (b) h = {q6b:.2f} h")


# ============================================================
# Question 13 — the Weekly data set
# ============================================================
Weekly = load_data("Weekly")
RESULTS["q13_shape"] = list(Weekly.shape)

# ------------------------------------------------------------ (a)
print("Q13(a): summaries")
desc = Weekly.describe()
RESULTS["q13a_describe"] = {
    c: {k: float(v) for k, v in desc[c].items()} for c in desc.columns
}
RESULTS["q13a_direction_counts"] = {
    k: int(v) for k, v in Weekly["Direction"].value_counts().items()
}
RESULTS["q13a_years"] = {
    "min": int(Weekly["Year"].min()),
    "max": int(Weekly["Year"].max()),
}

# Correlations between the numeric predictors
corr = Weekly.select_dtypes("number").corr()
RESULTS["q13a_corr"] = {
    c: {k: float(v) for k, v in corr[c].items()} for c in corr.columns
}
gui = Weekly[["Lag1", "Lag2", "Lag3", "Lag4", "Lag5", "Volume"]]

# --- Figure: boxplots of Lags (and Today) by Direction ---------------
fig, axes = plt.subplots(2, 3, figsize=(10.5, 6.2))
for ax, col in zip(axes.ravel(), ["Lag1", "Lag2", "Lag3", "Lag4", "Lag5", "Today"]):
    sns.boxplot(
        data=Weekly, x="Direction", y=col, hue="Direction", ax=ax, legend=False,
        palette=["#c44e52", "#4c72b0"], order=["Down", "Up"], width=0.55,
        fliersize=2.5,
    )
    ax.set_xlabel("")
    ax.set_title(f"{col} by Direction")
fig.tight_layout()
savefig(fig, "q13a_boxplots.png")

# --- Figure: volume growth over time ---------------------------------
fig, ax = plt.subplots(figsize=(9.2, 3.4))
ax.plot(np.arange(len(Weekly)), Weekly["Volume"], lw=0.7, color="#9fb3c8", alpha=0.9)
yearly = Weekly.groupby("Year")["Volume"].mean()
yearly_x = (yearly.index - 1990) * 52 + 26
ax.plot(yearly_x, yearly.values, color="#c44e52", lw=2.2, label="Yearly mean volume")
ax.set_xlabel("Year")
ax.set_ylabel("Volume")
ax.set_title("Weekly share volume grows over time")
ax.set_xticks(np.arange(0, len(Weekly) + 1, 104))
ax.set_xticklabels([str(1990 + w // 52) for w in np.arange(0, len(Weekly) + 1, 104)])
ax.legend()
fig.tight_layout()
savefig(fig, "q13a_volume.png")

# --- Figure: correlation heatmap -------------------------------------
fig, ax = plt.subplots(figsize=(6.4, 5.4))
sns.heatmap(
    corr, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1,
    square=True, cbar_kws={"shrink": 0.8}, ax=ax, annot_kws={"size": 8.5},
)
ax.set_title("Correlations among numeric Weekly variables")
ax.grid(False)
fig.tight_layout()
savefig(fig, "q13a_corr.png")

# ------------------------------------------------------------ (b)
print("Q13(b): logistic regression, full data, 5 lags + Volume")
preds_full = ["Lag1", "Lag2", "Lag3", "Lag4", "Lag5", "Volume"]
y_full = (Weekly["Direction"] == "Up").astype(int)
X_full = sm.add_constant(Weekly[preds_full], has_constant="add")
glm_full = sm.GLM(y_full, X_full, family=sm.families.Binomial()).fit()

tbl = glm_full.summary2().tables[1]
RESULTS["q13b_smry"] = {
    coef: {
```

```
        "coef": float(row["Coef."]),
        "se": float(row["Std.Err."]),
        "z": float(row["z"]),
        "p": float(row["P>|z|"]),
    }
    for coef, row in tbl.iterrows()
}
print(tbl.round(4).to_string())

# ------------------------------------------------------------------ (c)
yhat_prob_full = glm_full.predict(X_full)
yhat_full = np.where(yhat_prob_full > 0.5, "Up", "Down")
act_full = Weekly["Direction"].astype(str).to_numpy()

RESULTS["q13c_confusion"] = {
    "matrix": confusion_matrix(act_full, yhat_full, labels=["Down", "Up"]).tolist(),
    "labels": ["Down", "Up"],
    "n": int(len(act_full)),
    "correct": int((act_full == yhat_full).sum()),
    "frac": float((act_full == yhat_full).mean()),
    "down_correct_rate": float((yhat_full[act_full == "Down"] == "Down").mean()),
    "up_correct_rate": float((yhat_full[act_full == "Up"] == "Up").mean()),
}
print(f"  training fraction correct = {RESULTS['q13c_confusion']['frac']:.4f}")


# ---------- helpers for the (d)-(j) train/test experiments ------------
train_mask = Weekly["Year"] <= 2008
test_mask = Weekly["Year"] >= 2009


def fit_summary(
    method: str,
    predictors: list[str],
    Xtr: pd.DataFrame,
    ytr_bin: np.ndarray,
    ytr_lab: np.ndarray,
    Xte: pd.DataFrame,
    yte_lab: np.ndarray,
    k: int | None = None,
) -> dict:
    """Fit one of the five methods, return accuracy + confusion matrix."""
    if method == "Logit":
        fitG = sm.GLM(
            ytr_bin, sm.add_constant(Xtr, has_constant="add"),
            family=sm.families.Binomial(),
        ).fit()
        pr = fitG.predict(sm.add_constant(Xte, has_constant="add"))
        yh = np.where(pr > 0.5, "Up", "Down")
    elif method == "LDA":
        yh = LinearDiscriminantAnalysis().fit(Xtr, ytr_lab).predict(Xte)
    elif method == "QDA":
        yh = QuadraticDiscriminantAnalysis().fit(Xtr, ytr_lab).predict(Xte)
    elif method == "NB":
        yh = GaussianNB().fit(Xtr, ytr_lab).predict(Xte)
    elif method == "KNN":
        pipe = make_pipeline(
            StandardScaler(),
            KNeighborsClassifier(n_neighbors=k if k is not None else 1),
        ).fit(Xtr, ytr_lab)
        yh = pipe.predict(Xte)
    else:
        raise ValueError(method)
    cm = confusion_matrix(yte_lab, yh, labels=["Down", "Up"])
    out = {
        "method": method,
        "predictors": list(predictors),
        "matrix": cm.tolist(),
        "correct": int((yh == yte_lab).sum()),
        "n": int(len(yte_lab)),
        "frac": float((yh == yte_lab).mean()),
    }
    if k is not None:
        out["k"] = int(k)
    return out


# ------------------------------------------------- (d) through (h)
print("Q13(d-h): Lag2 only, train 1990-2008 / test 2009-2010")
lag2 = ["Lag2"]
Xtr_df = Weekly.loc[train_mask, lag2]
Xte_df = Weekly.loc[test_mask, lag2]
ytr_lab = Weekly.loc[train_mask, "Direction"].astype(str).to_numpy()
yte_lab = Weekly.loc[test_mask, "Direction"].astype(str).to_numpy()
ytr_bin = (Weekly.loc[train_mask, "Direction"] == "Up").astype(int).to_numpy()

results_lag2 = {
    m: fit_summary(m, lag2, Xtr_df, ytr_bin, ytr_lab, Xte_df, yte_lab, k=1 if m == "KNN" else None)
    for m in ["Logit", "LDA", "QDA", "KNN", "NB"]
}
RESULTS["q13dh_lag2"] = results_lag2
for m, r in results_lag2.items():
    print(f"  {m:5s}: {r['correct']:3d}/{r['n']} correct = {r['frac']:.4f}")
```

```python
# ------------------------------------------------------------ (j)
print("Q13(j): systematic experiment")
W = Weekly.copy()
W["Lag2_sq"] = W["Lag2"] ** 2
W["Lag1_sq"] = W["Lag1"] ** 2
W["Lag1Lag2"] = W["Lag1"] * W["Lag2"]
W["logVol"] = np.log(W["Volume"])

sets: list[list[str]] = [
    ["Lag2"],
    ["Lag1", "Lag2"],
    ["Lag2", "Lag3"],
    ["Lag1", "Lag2", "Lag3"],
    ["Lag2", "Lag5"],
    ["Lag1", "Lag2", "Lag3", "Lag4", "Lag5"],
    ["Lag1", "Lag2", "Lag3", "Lag4", "Lag5", "Volume"],
    ["Lag2", "Volume"],
    ["Lag2", "Lag2_sq"],
    ["Lag2", "Lag2_sq", "Lag1Lag2"],
    ["Lag1", "Lag2", "Lag1Lag2"],
    ["Lag1", "Lag2", "Lag1_sq", "Lag2_sq"],
    ["Lag1", "Lag2", "Volume"],
    ["Lag2", "logVol"],
    ["Lag1", "Lag2", "Lag3", "Lag4", "Lag5", "Volume", "Lag1Lag2"],
]
METHODS = ["Logit", "LDA", "QDA", "NB"]
KNNS = [1, 2, 3, 5, 7, 9, 10, 11, 12, 13, 14, 15, 17, 20, 25, 30, 47, 50]

exp_rows: list[dict] = []
for s in sets:
    Xtr = W.loc[train_mask, s]
    Xte = W.loc[test_mask, s]
    ytr_b = (W.loc[train_mask, "Direction"] == "Up").astype(int).to_numpy()
    ytr_l = W.loc[train_mask, "Direction"].astype(str).to_numpy()
    for m in METHODS:
        exp_rows.append(fit_summary(m, s, Xtr, ytr_b, ytr_l, Xte, yte_lab))
    for k in KNNS:
        exp_rows.append(fit_summary("KNN", s, Xtr, ytr_b, ytr_l, Xte, yte_lab, k=k))
    print(f"  done: {s}")

RESULTS["q13j_all"] = exp_rows
best = max(exp_rows, key=lambda r: r["frac"])
RESULTS["q13j_best"] = best
print(f"  BEST: {best['method']} {best['predictors']}"
      + (f" (K={best.get('k')})" if "k" in best else "")
      + f" -> {best['frac']:.4f}")

rank_table = []
for s in sets:
    row: dict = {"predictors": "+".join(s)}
    for m in METHODS:
        acc = next(r["frac"] for r in exp_rows if r["method"] == m and r["predictors"] == s)
        row[m] = acc
    knn_r = [r for r in exp_rows if r["method"] == "KNN" and r["predictors"] == s]
    best_knn = max(knn_r, key=lambda r: r["frac"])
    row["KNN"] = best_knn["frac"]
    row["K*"] = best_knn["k"]
    rank_table.append(row)
rank_table.sort(key=lambda r: -max(r[m] for m in METHODS + ["KNN"]))
RESULTS["q13j_rank_table"] = rank_table

# KNN K-sweep on Lag2 for the plot
K_VALUE = 1
knn_curve = []
for k in range(1, 51):
    r = fit_summary("KNN", lag2, Xtr_df, ytr_bin, ytr_lab, Xte_df, yte_lab, k=k)
    knn_curve.append({"k": k, "frac": r["frac"]})
RESULTS["q13j_knn_curve_lag2"] = knn_curve

fig, ax = plt.subplots(figsize=(7.2, 3.6))
ax.plot([r["k"] for r in knn_curve], [r["frac"] for r in knn_curve],
        marker="o", ms=3.5, lw=1.2, color="#4c72b0")
ax.set_xlabel("K (number of neighbors)")
ax.set_ylabel("Test accuracy")
ax.set_title("KNN on Lag2: test accuracy vs. K")
fig.tight_layout()
savefig(fig, "q13j_knn_sweep.png")

# ======================================================================
# Question 15 — Power, Power2, Power3, PlotPower
# ======================================================================
print("Q15: power functions")


# (a)
def Power() -> None:
    """Print 2**3."""
    print(2**3)


# (b)
def Power2(x, a) -> None:
    """Print x**a."""
    print(x**a)
```

```python
# (d)
def Power3(x, a):
    """Return x**a."""
    result = x**a
    return result


# (f)
def PlotPower(x: np.ndarray, a) -> plt.Figure:
    """Plot x**a over the vector x."""
    fig, ax = plt.subplots(figsize=(6.4, 4.2))
    ax.plot(x, Power3(x, a), marker="o", color="#4c72b0", lw=1.8, ms=4)
    ax.set_xlabel("x")
    if isinstance(a, int):
        ax.set_ylabel(f"$x^{a}$")
    else:
        ax.set_ylabel(r"$x^a$")
    ax.set_title(rf"$y = x^{a}$" if isinstance(a, int) else r"$y = x^a$")
    fig.tight_layout()
    return fig


print("(a) Power() output:")
Power()

RESULTS["q15"] = {
    "Power2_3_8": None,  # filled below (Power2 only prints)
    "c": {
        "10**3": int(Power3(10, 3)),
        "8**17": int(Power3(8, 17)),
        "131**3": int(Power3(131, 3)),
    },
}

print("(b) Power2(3, 8) output:")
Power2(3, 8)
RESULTS["q15"]["Power2_3_8"] = int(3**8)

print("(c) Power2 outputs:")
Power2(10, 3)
Power2(8, 17)
Power2(131, 3)

# (e) plot f(x) = x^2 on 1..10, linear + log-log panels
x10 = np.arange(1, 11)
fig, axes = plt.subplots(1, 2, figsize=(9.2, 3.8))
axes[0].plot(x10, Power3(x10, 2), marker="o", color="#4c72b0", lw=1.8, ms=4)
axes[0].set_xlabel("$x$")
axes[0].set_ylabel("$x^2$")
axes[0].set_title(r"$f(x) = x^2$ (linear scale)")
axes[1].plot(x10, Power3(x10, 2), marker="o", color="#c44e52", lw=1.8, ms=4)
axes[1].set_xscale("log")
axes[1].set_yscale("log")
axes[1].set_xlabel(r"$x$ (log scale)")
axes[1].set_ylabel(r"$x^2$ (log scale)")
axes[1].set_title(r"$f(x) = x^2$ (log-log scale)")
fig.tight_layout()
savefig(fig, "q15_power3.png")

# (f) PlotPower(np.arange(1, 11), 3)
fig = PlotPower(np.arange(1, 11), 3)
savefig(fig, "q15_plotpower.png")

# ============================================================================
# dump results
# ============================================================================
with open(os.path.join(HERE, "results.json"), "w") as fh:
    json.dump(RESULTS, fh, indent=1, default=str)
print("wrote results.json")
```